

Integrazione pagAmico

Librerie C# e Kotlin per la cassa rendiresto PayPrint
e comunicazione con il dispositivo e con il simulatore

Contenuto	Libreria C# (net8.0, netstandard2.0, net47), libreria Kotlin (JVM/Android), banchi di prova, collaudo automatico, proxy di analisi, riempimento del simulatore
Protocollo	PayPrint pagAmico, TCP-IP rev. 2.33 - firmware 8.72
Documenti di base	Manuale Lista comandi TCP-IP 2.33; Integrazione display 1.2; Protocollo di stampa 2.00; Note di rilascio FW 8.72; Guida pagAmico Dev Kit 1.0
Stato della verifica	168 test offline e 64 passi di collaudo sul simulatore, superati in entrambi i linguaggi (11 settembre 2026)

A cosa serve questo documento

Spiega come sono fatte le due librerie, che cosa succede sul filo quando l'applicazione chiede un incasso, e come si collauda tutto contro il simulatore fornito da PayPrint senza avere la macchina in ufficio. Il capitolo 9 aggiunge il passo successivo: che cosa comporta innestare la libreria in un gestionale che pilota già una cassa automatica di un'altra marca. Le parti segnalate come *da confermare* sono punti in cui la documentazione del costruttore si contraddice o tace.

1. Che cosa contiene il pacchetto

Il pacchetto e' diviso in due alberi indipendenti, uno per linguaggio. Le due librerie espongono le stesse funzionalita' con gli stessi nomi, adattati alle convenzioni di ciascun linguaggio: chi conosce una conosce l'altra.

Lato C#

Progetto	Ruolo
PayPrint.PagAmico	la libreria: nessuna dipendenza esterna, solo System.Text.Json. Compila per net8.0, netstandard2.0 e net47
PayPrint.PagAmico.WinForms	banco di prova con tutti i comandi, pannello del traffico e finestra di log
PayPrint.PagAmico.Tests	168 test offline: 86 confrontano le stringhe generate con gli esempi dei manuali, 62 fanno parlare il client con un finto pagAmico locale, 20 provano il registro su file e gli errori
PayPrint.PagAmico.LiveTest	collaudo end-to-end: 64 passi in 13 gruppi di default (71 in 15, accendendo la sonda DI e i riavvii), contro simulatore o macchina reale
PayPrint.PagAmico.Tap	proxy TCP che registra il traffico fra un client qualsiasi e la macchina
PayPrint.PagAmico.Fill	riempie le giacenze del simulatore prima di una sessione di prove
PayPrint.PagAmico.Demo	esempio minimo da riga di comando

Lato Kotlin

Modulo	Ruolo
pagamico-lib	la libreria (coroutine + kotlin-serialization), gli stessi 168 test offline e lo stesso collaudo a 64 passi
pagamico-desktop	banco di prova Compose for Desktop, gemello di quello WinForms

Portabilita' su Android

La libreria Kotlin usa solo `java.net.Socket`, `java.math.BigDecimal` e `java.time`: funziona su Android con `minSdk 26` (oppure con il core library desugaring), il permesso `INTERNET` e `kotlinx-coroutines-android`. Non serve il plugin di serializzazione, perche' si usa solo l'API `JsonElement` a runtime.

2. Il protocollo in una pagina

Il pagAmico e' un server TCP. Il client apre **una sola connessione** e la tiene aperta: porta di fabbrica 9100, che il manuale consiglia di cambiare perche' e' la porta di stampa RAW standard e spesso e' gia' occupata.

Aspetto	Come funziona
Comando	stringa ASCII a lunghezza fissa, chiusa da CR o CR+LF secondo PayPrint (il manuale non lo dice e i suoi esempi ne sono privi; le librerie oggi inviano senza): IN001050 chiede un incasso di 10,50 euro
Importi in richiesta	centesimi , con un numero di cifre fisso per comando (6 per IN, 10 per PA)
Importi in risposta	euro come numero JSON: "collectedAmount": 10.5
Risposta	oggetto JSON con sempre gli stessi campi; cambia il valore di response

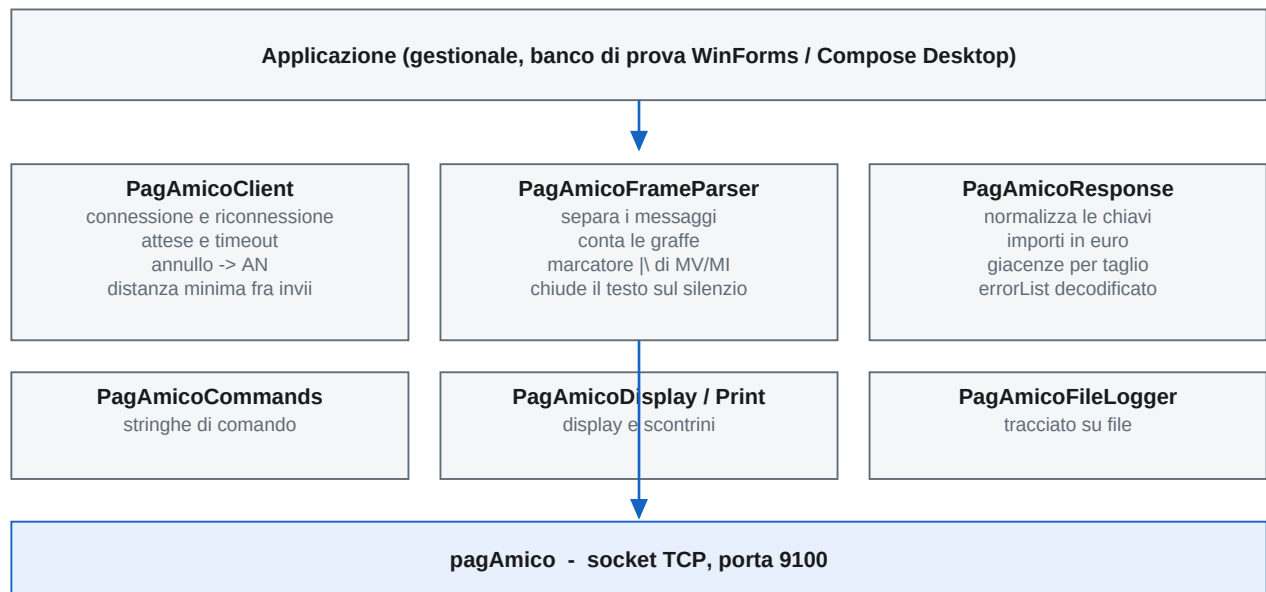
Aspetto	Come funziona
Asincronia	a un comando seguono piu' messaggi: accettazione, parziali, esito finale
Risposte non JSON	testo puro: CMD ERROR, OK STST, BT1, AN, EX, il contenuto di un barcode letto
Risposte lunghe	MV e MI possono superare il buffer del socket e terminano con il marcatore \
Comandi muti	CL, DS, DC, QA, CO, DT, DG, TS non rispondono nulla: attendere una risposta li' significa restare appesi

La regola che il manuale non scrive

Ogni comando va chiuso da **CR** o **CR+LF**: lo dice PayPrint, il manuale no. Sul simulatore, senza terminatore, il buffer del socket viene interpretato come **un solo comando**: due comandi trasmessi a distanza quasi nulla vengono letti insieme e il secondo viene ignorato, senza alcuna risposta e senza errore. Misurato sul simulatore: a 0 ms il comando si perde, a 30 ms passa. Le librerie, che oggi inviano senza terminatore, impongono perciò una distanza minima fra gli invii, 80 ms per default. PayPrint dice la pausa non necessaria con il terminatore: resta come rete di sicurezza finché la prova col CR non la rende superflua (docs/esito-risposta-payprint.md).

3. Architettura delle librerie

La stessa struttura in C# e in Kotlin. L'applicazione parla solo con PagAmicoClient; tutto il resto e' interno o di supporto.



I blocchi della libreria e il loro rapporto con l'applicazione e con la macchina.

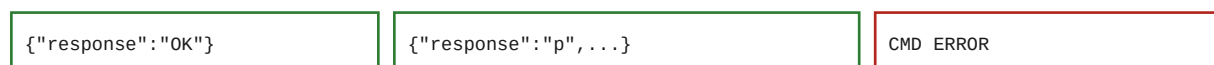
3.1 Il separatore dei messaggi

E' la parte piu' delicata, perche' il protocollo non ha un terminatore unico. Il TCP puo' spezzare un JSON su piu' letture oppure accorparne due nella stessa, e in mezzo possono arrivare risposte testuali senza alcun delimitatore. Il separatore lavora cosi':

Byte in arrivo dal socket (tre letture successive):



Messaggi consegnati all'applicazione:



i primi due riconosciuti contando le graffe, il terzo (testo puro) chiuso dopo 150 ms di silenzio

Dal flusso di byte ai messaggi: graffe contate per il JSON, silenzio per il testo.

Situazione	Come viene risolta
messaggio JSON	conteggio delle graffe, ignorando quelle dentro le stringhe: lo scontrino POS contiene graffe, virgolette e backslash
JSON incompleto	resta nel buffer finche' non arriva il resto
due JSON insieme	vengono separati e consegnati uno alla volta
risposta MV / MI	il marcatore \ viene riconosciuto e rimosso
testo puro	chiuso su CR/LF, oppure dopo 150 ms di silenzio, oppure quando inizia un JSON
JSON mai completato	dopo 10 secondi viene emesso come testo grezzo, per non restare bloccati

3.2 Attese, parziali e annullo

Ogni comando registra un'attesa con un predicato che riconosce il messaggio finale. I messaggi intermedi non vengono buttati: sono consegnati all'applicazione come avanzamento, così la cassa può mostrare l'importo che sale mentre il cliente inserisce le monete. Se l'applicazione annulla l'operazione, la libreria non abbandona l'attesa: manda AN alla macchina e resta in ascolto dell'esito, perché l'annullo comporta la restituzione del denaro già inserito e quel dato serve.

3.3 Nomi dei campi instabili fra firmware

Il firmware 8.71 ha accorciato alcuni nomi per far stare il JSON in un unico blocco. Le chiavi vengono perciò normalizzate (minuscolo, senza underscore e senza spazi) e mappate su più alias, così lo stesso codice funziona con le macchine vecchie e nuove.

Campo	Varianti gestite
numero di serie	sN (da 8.71) e serialNumber
id del movimento	Id (da 8.71) e OperationId
importo trattenuto (controllo)	committedAmount e il refuso committedAmout; l'importo si legge in collectedAmount
scontrino POS	posFinancial... e PosFinancial...
azzeramento BTA	" AmountResettedBanknotesInBTA", con lo spazio iniziale presente nel firmware

3.4 Un comando alla volta

I comandi sono **serializzati su un mutex**: la libreria ne tiene in volo uno solo per connessione. Serve a non mescolare le risposte, dato che il protocollo non le correla alla richiesta in nessun modo. La pausa minima fra un invio e il successivo (sezione 8.1) è invece imposta da un **secondo** semaforo, quello di scrittura, che avvolge ogni singolo invio: vale anche per i comandi che il mutex non lo prendono mai. Sono due meccanismi distinti, e conviene non confonderli: togliendo il mutex la pausa resterebbe.

La conseguenza da conoscere prima di progettarci sopra

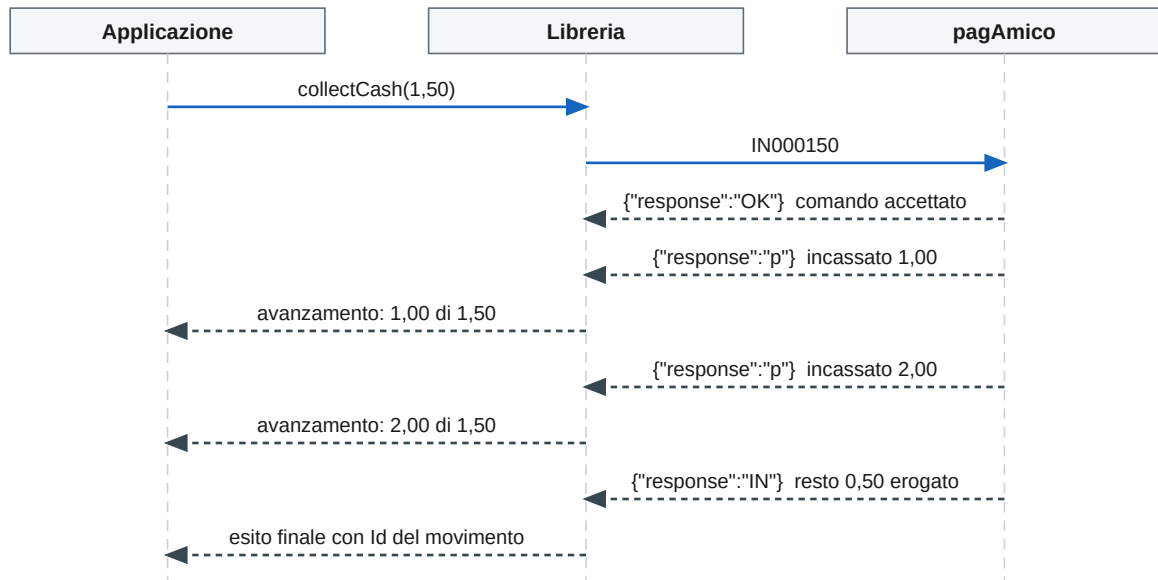
L'incasso **tiene il mutex per tutta la sua durata**, fino al timeout di transazione. Durante IN la macchina accetta solo AN e CM, e agli altri risponde BUSY. Dall'11 settembre la libreria lo fa rispettare: finché `IsCollecting` è vero, ogni altro invio - stato, display, immagini, stampa - **lancia subito** `PagAmicoCollectionOpenException` senza trasmettere nulla. Commit e annullo passano per una **via laterale** fuori dal mutex: partono dopo l'OK di accettazione, e ne parte uno solo per incasso. Fino al 10 settembre si accodavano dietro l'incasso, e un display partiva davvero provocando un BUSY che chiudeva l'attesa (difetti D1 e D3, docs/esito-risposta-payprint.md).

Chi integra deve quindi decidere **chi possiede la connessione** e che cosa succede se un secondo interessato - un controllo periodico di presenza, una pagina di diagnostica, il display - prova a parlare con lo stesso box mentre un incasso è aperto. Su una API senza stato la domanda non si pone; qui sì.

4. Che cosa succede sul filo

4.1 Incasso in contanti

Il caso piu' importante. L'applicazione chiede 1,50 euro; il cliente inserisce due monete da 1 euro; la macchina eroga 0,50 di resto e chiude.

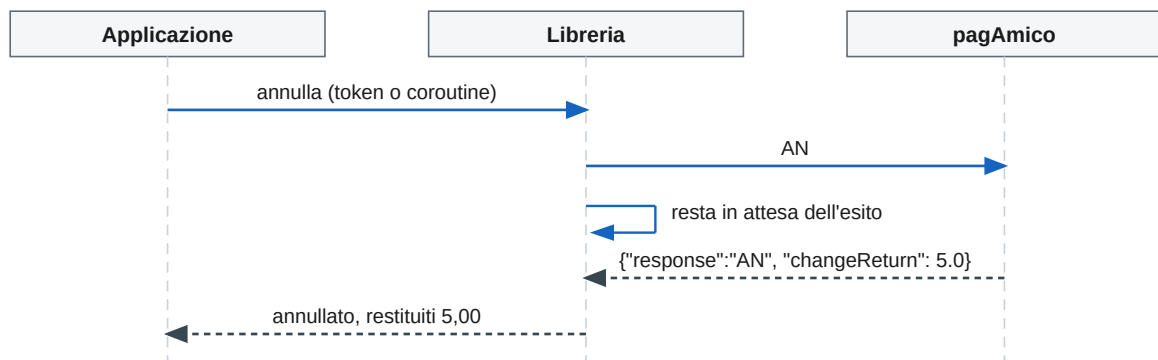


Un comando, quattro risposte. La libreria consegna i parziali e restituisce solo l'esito.

Il campo da non ignorare mai

Se la macchina non riesce a comporre il resto per mancanza di un taglio, l'incasso si chiude comunque e la differenza finisce in `amountUnpaid`. Il denaro resta dovuto al cliente: va segnalato all'operatore, non trattato come una transazione riuscita.

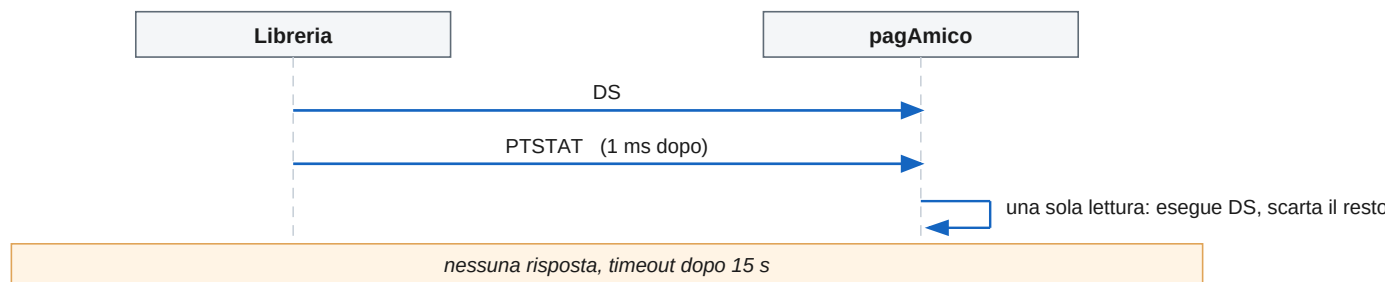
4.2 Annullo dell'incasso



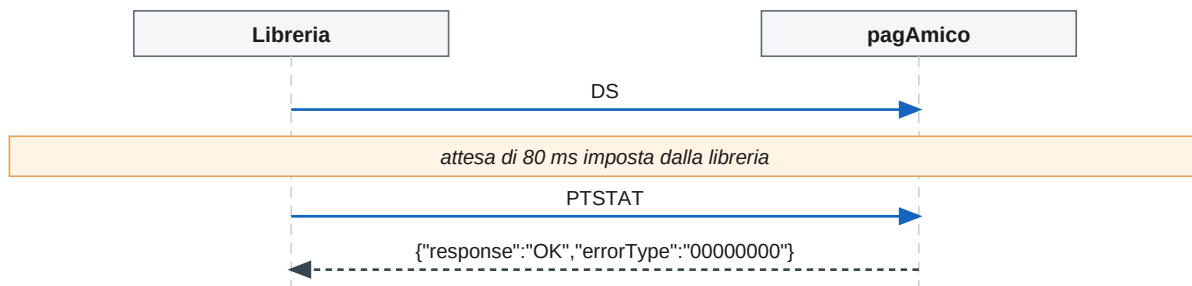
La cancellazione non interrompe l'attesa: serve sapere quanto e' stato restituito.

4.3 Il difetto scoperto in collaudo

Sequenza usata nel banco di prova: chiudere la finestra di testo e subito dopo chiedere lo stato della stampante. Nella prima versione, sul simulatore e con i comandi inviati senza terminatore, l'attesa scadeva dopo 15 secondi senza che arrivasse nulla.



Prima: i due comandi arrivano insieme e il secondo sparisce.



Dopo: con la distanza minima fra invii il simulatore risponde regolarmente. Il rimedio indicato da PayPrint e' il terminatore CR o CR+LF: la pausa resta come rete di sicurezza finche' non e' provato.

5. Come si usa

C#

```
using var client = new PagAmicoClient("192.168.1.231", 9100);
await client.ConnectAsync();

var stato = await client.GetStatusAsync(); // [ST]
Console.WriteLine(stato.BanknotesAvailableByDenomination[20]);

var esito = await client.CollectCashAsync( // [IN]
    10.50m,
    new Progress<PagAmicoResponse>(p =>
        Console.WriteLine($"incassato {p.CollectedException}")));

if (esito.AmountUnpaid > 0)
    Console.WriteLine($"resto non erogato: {esito.AmountUnpaid} EUR");
```

Kotlin

```
val client = PagAmicoClient("192.168.1.231", 9100)
client.connect()

val stato = client.status() // [ST]
println(stato.banknotesAvailableByDenomination[20])

val esito = client.collectCash(BigDecimal("10.50")) { p -> // [IN]
    println("incassato ${p.collectedAmount}")
}

esito.amountUnpaid?.takeIf { it.signum() > 0 }?.let {
    println("resto non erogato: $it EUR")
}
```

5.1 I comandi coperti

Area	Comandi
Incasso	IN, I2, P0, IM, AN, CM
Erogazione	PA, P2, PM
Contanti e scorte	ST, M2, MF, BT, AZ, AF, SM, SB, EM, EB
Ricariche	RC, RS, VC, VS, RM, R3, RB, R2, FR
POS	PL, PR, PS, PZ, PP
Sistema e movimenti	CL, RI, L0, MV, MI
Display	DT, DG, DS, DM, DC, DI, QR, QA, TS, ID, CO
Immagini	SF, SI, SR
Stampa	tutti i comandi PT del manuale 2.00, piu' un compositore di scontrini

Il manuale indica come minimo indispensabile i comandi IN, P0, AN, PA, RI, ST, CL, RS, RC, il controllo delle soglie minime e la gestione di `errorList` ed `errorCode`: tutto coperto.

5.2 Le due giacenze da non confondere

La macchina espone due grandezze diverse che durante l'esercizio si scollano. La **giacenza reale** e' quello che c'e' fisicamente dentro e decide se un resto e' erogabile. Il **fondo cassa** e' un valore contabile che cambia solo quando si manda AF, e serve per la quadratura. La libreria le tiene separate.

	giacenza reale	fondo cassa
monete	CoinsByDenomination	CoinsInStockByDenomination
banconote	BanknotesAvailableByDenomination	BankNotesInStockByDenomination

La trappola dei cassetti

Le banconote possono essere caricate in ordine qualsiasi e lo stesso taglio puo' stare in piu' cassetti. Il taglio si legge dalla posizione 0 di ogni cassetto, mai dedotto dalla posizione del cassetto nell'array. BanknotesAvailableByDenomination fa la somma nel modo corretto: usare quella.

5.3 Log e diagnostica

Quando un comando non risponde, il traffico grezzo non basta a capire perche': mostra i byte usciti, non se il client li ha trattiene, se il messaggio ricevuto era completo o se l'attesa e' semplicemente scaduta. Per questo la libreria espone tre livelli di traccia indipendenti, attivabili uno per volta.

Livello	C#	Kotlin	Che cosa registra
traffico	CommandSent, MessageReceived	messages	le stringhe inviate e i messaggi ricevuti, cosi' come sono
diagnostica	Trace	onTrace	connessione, pause imposte fra un invio e l'altro, byte letti dal socket, separazione dei messaggi, resto rimasto nel buffer, attese soddisfatte o scadute, messaggi non sollecitati
su file	PagAmicoFileLogger	PagAmicoFileLogger	un file al giorno con tutto quanto sopra, con orario al millisecondo

Una sessione vista dalla diagnostica:

```
.. connesso a 127.0.0.1:9100 (pausa minima fra invii 80 ms, terminatore nessuno)
.. attesa di 80 ms prima dell'invio: il pagAmico ignora i comandi troppo ravvicinati
.. in attesa dell'esito di 'ST' (timeout 15s)
.. letti 1076 byte dal socket
.. messaggio separato: JSON, 1076 caratteri
.. attesa di 'ST' soddisfatta da: {"response":"ST", ...}
```

Le righe che contano davvero sono le ultime due quando **non** compaiono: se la traccia si ferma su *in attesa* e poi dice *attesa scaduta* senza aver letto un solo byte, il comando non e' mai arrivato alla macchina. E' esattamente il sintomo del difetto descritto al capitolo 8.

I due banchi di prova hanno le stesse caselle: *registra su file*, *diagnostica libreria* e una *finestra traffico* separata, con filtri per direzione, ricerca testuale, scorrimento automatico ed esportazione.

Uso su .NET Framework

La libreria C# compila anche per netstandard2.0 e net47, quindi entra in un progetto legacy senza riscritture: il codice moderno resta dentro la libreria e verso l'esterno espone solo tipi disponibili ovunque. Su un progetto con packages.config vanno pero' aggiunte a mano le dipendenze transitive di System.Text.Json, che in quel formato non arrivano da sole.

6. Errori e stato macchina

Le anomalie viaggiano su tre campi distinti, che vanno letti insieme.

Campo	Quando	Significato
errorList	sempre	fotografia dello stato della macchina, anche a comando riuscito
errorCode	con response = ER	perche' il comando non e' andato a buon fine
errorType	insieme a errorCode	la circostanza: stato del dispositivo, esito, codice del POS

6.1 errorList: quattro cifre dopo la E

Posizione	Valori
1 - pagamento	0 ok - 1 importo superiore alla disponibilita' di monete - 2 importo non erogabile
2 - monete	0 ok - 1 sottoscorta - 2 troppe monete - 9 esaurite
3 - banconote	0 ok - 1 sottoscorta - 2 troppe - 5 entrambi i cassetti vuoti - 6 un cassetto vuoto - 9 un taglio esaurito
4 - cassetto BTA	0 ok - 1 prossimo al riempimento - 2 pieno

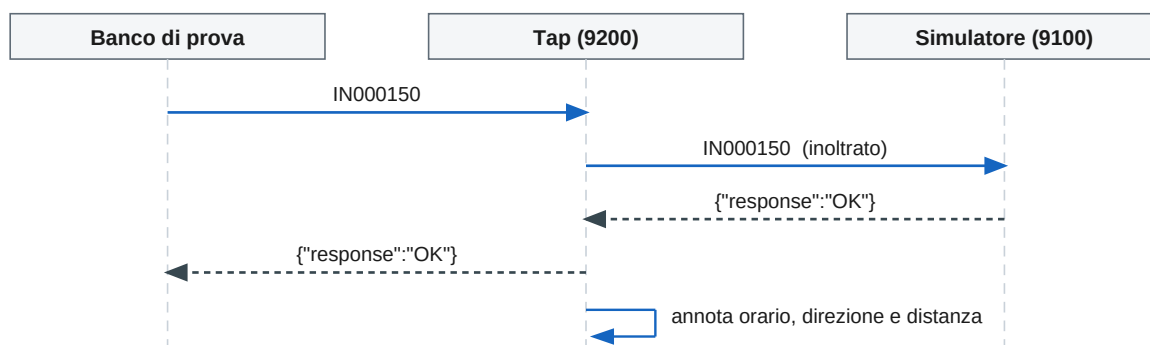
Le librerie lo decodificano in avvisi leggibili. Il manuale indica come imprescindibili il controllo delle soglie minime e la gestione della condizione *troppe monete*: senza quelli la cassa arriva a non erogare piu' il resto.

6.2 I codici piu' frequenti

Codice	Significato
E100	comando non eseguibile ora: errorType dice perche' (0 fuori servizio, 1 ok, 2 avvio, 3 setup, 5 reboot, 99 occupato)
E200 / E201 / E202	accettatori vuoti, stacker banconote vuoto, hopper monete vuoto
E300 / E301	importo oltre il limite erogabile, oppure oltre la disponibilita'
E302 / E303	banconote o monete insufficienti per comporre il pagamento
E500	il POS non risponde o ha rifiutato: errorType contiene il codice Ingenico
NOT_STARTED	accettatori guasti: dal FW 8.71 la macchina riparte in modalita' emergenza, solo POS

7. Il simulatore e gli strumenti di prova

PayPrint distribuisce **pagAmico Dev Kit**, un'applicazione Windows che contiene un pagAmico emulato: apre un socket TCP, accetta i comandi documentati e risponde con gli stessi JSON del dispositivo reale, comprese le giacenze che si aggiornano a ogni operazione. Si avvia dalla sezione *Simulatore*, poi ci si collega come a una macchina vera.



Il proxy di analisi si inserisce fra client e macchina senza modificare nulla.

Strumento	A che serve
Collaudo automatico	64 passi in sequenza, divisi in gruppi selezionabili, con esito per ciascuno. Esercita tutti i comandi implementati
Proxy di analisi	registra il traffico fra un client e la macchina, segmento per segmento. Abbina ogni risposta al comando che l'ha chiesta, misura le pause del client e i tempi di risposta della macchina, segnala gli invii troppo ravvicinati. Puntandoci il Dev Kit di PayPrint si vede cosa manda la loro applicazione: e' il modo per chiudere alcuni dei punti ancora aperti (sezione 8.2)
Riempimento	il simulatore si svuota dopo qualche prova. Le sessioni di ricarica non aggiungono nulla, perche' sulla macchina vera il contante lo mette l'operatore: lo strumento riempie eseguendo incassi, alternando i tagli
Log su file	un file al giorno con tutto il traffico. Il Dev Kit mostra il traffico a video ma non lo salva

7.1 I gruppi del collaudo

Senza argomenti il collaudo esegue tutti i gruppi tranne riavvii. Indicandone uno si prova solo quello, utile quando si sta lavorando su un'area sola.

Gruppo	Copre
base	ST e decodifica delle giacenze, CL, LO
cash	SM, SB, EM, EB, AF, BT
collect	IN con parziali, annullo dal client, CM, PA
erogazione	P2, PM, M2, MF, AZ, I2, IM
ricariche	RM, R3, RB, R2, RS, VC, VS, ciascuna chiusa con FR
display	DT/DS, DM/DC, QR/QA, TS/ID/CO
display2	DG, DI con tastiera numerica
print	PTSTAT e uno scontrino completo da PTSTST a PTSTEN
print2	PTITON/OF, PTDBON/OF, PTUN, PTFB, PTCP, PTPRWR, PTPRRE, PTBCBC, PTPRDT, PTSTAN
system	PR, PL, PO, RC/FR, e un comando inesistente per verificare CMD ERROR
pos2	PLT con ristampa, PS, PP
immagini	SI, SR, SF, e il confronto fra i due incapsulamenti binari
movimenti	MV, MI
riavvii	PZ e RI: esclusi di default, riavviano POS e macchina

I passi che erogano o spostano banconote leggono prima ST e scelgono un taglio effettivamente presente: dopo un'erogazione quel taglio potrebbe non esserci piu', e un collaudo scritto su valori fissi fallirebbe per colpa propria.

Che cosa il simulatore non riproduce

Il cliente virtuale si attiva solo sugli incassi: durante una sessione di ricarica (RC, RS, RM, RB) non entra denaro, perché lì il contante lo inserisce fisicamente l'operatore. Verificato: RS seguito da FR lascia le giacenze identiche. Nella sezione *Simulatore* va acceso l'interruttore *Il cliente inserisce l'importo esatto*: senza, il cliente arrotonda per eccesso e la macchina restituisce come resto quasi tutto quello che incassa.

7.2 Le tabelle del catalogo comandi

Il Dev Kit e' un'applicazione Flutter e la sua logica Dart e' compilata in data/app.so. Il file non e' decompilabile, ma conserva i letterali di stringa, e fra questi c'e' il catalogo comandi che l'applicazione mostra a video: descrizioni, esempi e tabelle scritti da PayPrint. Le tabelle che seguono vengono da li' e dalla *Guida all'uso* che accompagna il Dev Kit, un documento a se' che riassume i quattro manuali PayPrint e in piu' punti li disambigua. Coprono punti che i manuali da soli lasciavano aperti.

Tipi di codice a barre (terzo carattere di PTBCBC)

Cod.	Tipo	Cod.	Tipo
0	UPC-A, 12 caratteri	5	ITF
1	UPC-E, 8 caratteri	6	CODEBAR
2	EAN13, 13 caratteri	7	CODE93
3	EAN8, 8 caratteri	8	CODE128
4	CODE39		

Chiude la contraddizione fra tabella comandi e note di rilascio sui codici 7 e 8. I tipi a lunghezza fissa rifiutano stringhe di lunghezza diversa.

Causali dei movimenti (ultime tre cifre di MV, e campo codice_operazione nella risposta)

Cod.	Significato	Cod.	Significato
000	tutte le causali	041	scarico banconote
001	incasso	051	scarico banconote in BTA
010	pagamento	061	scarico banconote
011	scarico monete	071	scarico monete
021	ricarica monete	090	incasso POS
031	ricarica banconote		

I doppioni sono del produttore, non nostri: nel suo catalogo 011 e 071 portano la stessa etichetta, e cosi' 041 e 061. Resta da farsi dire che differenza c'e'.

Attributi del display (validi per DT, DG, DM, DI, QR)

Campo	Valori
colore font	0 blu, 1 nero, 2 verde, 3 rosso, 4 magenta, 5 ciano, 6 bianco, 7 grigio, 8 grigio scuro, 9 nero
stile font	0 normale, 1 grassetto, 2 corsivo, 3 grassetto corsivo
tastiera (DI)	0 nascosta, 1 alfanumerica, 2 numerica senza decimali, 3 numerica con decimali
modo lettura (QR)	0 con tastiera, 1 senza tastiera, 2 tessera sanitaria dal POS, 3 inserimento tessera, 4 tastiera numerica
allineamento	0 sinistra, 1 centro, 2 destra
etichette dei bottoni	un'etichetta vuota nasconde il pulsante
sottolineatura (stampa)	00 nessuna, 01 livello 1, 02 livello 2
testo del barcode	0 nessuno, 1 sopra, 2 sotto, 3 entrambi

Le due tastiere non vanno confuse: quella di DI parte da 0 = nascosta, quella di QR da 0 = con tastiera. Le librerie le tengono in due enumerativi distinti, KeyboardLayout e KeyboardMode.

Il formato di DI, finalmente

L'esempio del produttore ha dodici campi:

DI|16|0|1|SCRIVI NOME|18|1|0|OK|ANNULLA|ESCI|3

dimensione, stile e colore del titolo; il titolo; gli stessi tre attributi per il campo di input; le tre etichette dei bottoni; il tipo di tastiera. Le librerie ne inviano sette e il simulatore accetta lo stesso, ma così non si vedono né i bottoni né la formattazione del campo. La variante a dodici campi è stata aggiunta al gruppo di collaudo: va confermata sulla macchina vera prima di cambiare il metodo pubblico.

Altre conferme

Punto	Esito
PTPRDT_ASCII:	il manuale alternava tre grafie, il Dev Kit usa questa: è quella che inviamo
DT, DG, DM, QR	gli esempi del produttore coincidono con le nostre stringhe carattere per carattere
EM / EB	sigla più dodici cifre, due per taglio: formato confermato
codici di errore	il Dev Kit conosce DISPAG, ERPWD PAG, ERRP WD PAG, ERR LUNGHEZZA, ERR NON DISP, ERR SCOMAX, QTAMONETE, QTABANCONOTE: la nostra tabella li ha già tutti
immagini	rettangolo consigliato 571x520 dip. I delimitatori del payload binario restano non documentati
marcatore \	le risposte di MV e MI possono superare il buffer del socket e finiscono con questo marcatore: il parser lo riconosce e lo consuma
risposte di stampa	in chiaro, non JSON: OK <comando>, ER BUSY, ER N0-PRINT, ER CMD-ERROR. Gestite tutte
porta 9100	è quella di fabbrica ma è la porta RAW di stampa, molto usata: PayPrint consiglia di cambiarla nel Setup del pagAmico, ad esempio 43775
P2 risponde PB	non è una stranezza del firmware: il manuale 2.33 lo scrive a pagina 37, la risposta positiva di P2 porta il tag del comando deprecato. Le librerie accettano entrambi
tipi di barcode	il manuale Protocollo di stampa 2.00 si contraddice da solo: pagina 3 dice 7 = CODE128 e 8 = CODE93, la sua nota di aggiornamento a pagina 5 dice l'opposto. Il Dev Kit conferma la seconda, ed è quella implementata

8. Esito delle verifiche

I test offline confrontano le stringhe generate con gli esempi **letterali** dei manuali (IN001050, PA0000001050mypassword, PM001001000000000000, SM005100005100005100005100005100005100, EM010101011111, MV2025/01/01 10:002025/04/22 18:59000, PTQRQR42www.pagamico.it, PTBCBC210029876543210123, QR|...|16|1|2|0). Il collaudo invece parla con una macchina.

Verifica	C#	Kotlin
test offline sugli esempi dei manuali	86 su 86	86 su 86
test offline contro un finto pagAmico (incasso, comandi semplici, invio)	62 su 62	62 su 62
test offline sul registro su file e sugli errori	20 su 20	20 su 20
collaudo sul simulatore	64 su 64	64 su 64

I 64 sono i passi dei **gruppi di default**. Non coincidono con le 49 chiamate presenti nel sorgente, perché tre gruppi girano in ciclo: le sette sessioni di ricarica partono da un solo punto, i comandi di stampa singoli da un altro, e altrettanto la sonda del comando non documentato. Accendendo anche quella sonda e i riavvii i passi diventano **71**.

Il collaudo esercita **tutti** i comandi implementati: restano fuori i soli PB e MB, che il manuale marca come deprecati e sostituisce con P2 e M2.

8.1 Difetti trovati grazie al collaudo

Difetto	Effetto	Rimedio
comandi inviati troppo ravvicinati	sul simulatore, senza terminatore, il secondo veniva ignorato senza risposta; l'attesa scadeva dopo 15 o 45 secondi	distanza minima fra invii, 80 ms per default. PayPrint indica il terminatore CR o CR+LF, senza pause: la pausa resta come rete di sicurezza finché non è provato
Kotlin: cancellazione confusa con timeout	annullando un incasso il comando AN non partiva e la macchina restava occupata per i comandi successivi	la cancellazione del chiamante viene propagata come tale

8.2 Punti da confermare con PayPrint

L'elenco si è accorciato da una parte e allungato dall'altra. Il catalogo comandi estratto dal Dev Kit (sezione 7.2) ha chiuso i tipi di barcode, la grafia di PTPRDT_ASCII: e le tabelle degli attributi del display. In compenso l'analisi dell'innesto nel gestionale (capitolo 9) ne ha aperti sei nuovi, di natura diversa: non sono ambiguità dei manuali, sono **comportamenti della macchina che il simulatore non permette di osservare**. La risposta di PayPrint dell'11 settembre 2026 ha chiuso il terminatore, CM durante l'incasso e i parziali, e chiarito solo in parte pausa, messaggi spontanei e incasso abbandonato; amountPaid lo definiva già il manuale. Lo stato di ciascun punto è nella tabella; l'analisi, i difetti emersi e le domande nuove stanno in docs/esito-risposta-payprint.md.

Punto	Situazione
distanza minima fra comandi	chiarita in parte: per PayPrint nessuna pausa è necessaria se i comandi terminano con CR o CR+LF, e il simulatore <i>probabilmente ha qualche difficoltà</i> . Gli 80 ms sono una misura empirica sul simulatore senza terminatore e restano come rete di sicurezza finché una raffica con CR non regge sulla macchina
terminatore dei comandi	chiuso per la macchina: CR o CR+LF, dice PayPrint. Il manuale non lo nomina e i suoi esempi ne sono privi (pp. 12, 15, 43, 66). Il client oggi invia senza terminatore e il simulatore accetta; l'opzione per aggiungere CR/LF c'è, il default va portato a CR dopo averlo provato sul simulatore

Punto	Situazione
comando DI	assente dai manuali. Il Dev Kit ne mostra un esempio a dodici campi, molto piu' ricco dei sette che inviamo: da confermare sulla macchina vera, il simulatore accetta entrambi
comando ID	il manuale scrive TD<JSON> come formato completo mentre il comando e' ID
invio immagini	il manuale e il suo esempio Python indicano due incapsulamenti diversi, il Dev Kit dice solo <i>fra i delimitatori documentati</i> : sono implementati entrambi
causali dei movimenti	confermato che 041 e 061 valgono entrambe <i>scarico banconote</i> e 011 e 071 <i>scarico monete</i> : resta da capire che differenza c'e'
stampa immagine IM	presente nel riepilogo dei comandi ma senza sintassi: non implementato
CM durante un incasso	chiuso: PayPrint conferma che durante IN la macchina accetta solo AN e CM; ST va mandato solo a transazione chiusa, prima verrebbe ignorato. Il limite nella libreria e' tolto l'11 settembre: il CM a incasso aperto passa per una via laterale e il collaudo usa l'incasso di libreria. Da chiedere quanti frame seguono CM: sul simulatore due, e la libreria ora chiude sul secondo (difetto D1, corretto)
campi della risposta AN	ancora aperto. Arrivano collectedAmount e amountUnpaid, o solo changeReturn? Senza il primo non si sa se la restituzione e' stata incompleta , cioe' se e' rimasto in macchina denaro del cliente. PayPrint ha spiegato amountUnpaid (resto non erogato per mancanza di monete), ma la frase riguarda il resto di un IN (manuale p. 16), non il rimborso
contenuto del parziale p	chiuso: PayPrint dice che i parziali arrivano a ogni aggiunta di contante e sono cumulativi , quindi uno perso non costa nulla: il successivo porta il totale
semantica di amountPaid	chiuso dal manuale, che lo definisce <i>Importo erogato totale</i> (pp. 9, 59). Nel simulatore vale l'erogato di PA ed e' 0 sul resto di un IN: da confermare se comprende il resto, che oggi va sommato da changeCoins + changeBanknotes
messaggi spontanei	chiarito in parte: CMD ERROR risponde a un comando sconosciuto, BUSY a un comando che la macchina impegnata non puo' eseguire. Sono risposte a comandi nostri, non messaggi spontanei; su barcode, BT1 ed EX PayPrint non dice nulla. Il rischio era concreto - un BUSY provocato da un nostro invio a incasso aperto chiudeva l'attesa (difetto D3) - ed e' chiuso l'11 settembre: gli invii laterali sono bloccati, e dopo l'accettazione un testo non tocca l'incasso
incasso abbandonato	chiarito in parte. Dopo IN la macchina non ha timeout e l'incasso resta aperto; PayPrint dice di non metterne uno (I2 c'e' ma lo sconsiglia), mentre la libreria allo scadere dei suoi 5 minuti abbandona l'incasso senza mandare AN (difetto D2). Dopo una caduta di rete la macchina accetta la riconnessione dallo stesso IP; dal pannello si chiude con una chiusura forzata (RESTO + password) che restituisce AN. Resta da sapere se i frame arrivano sul nuovo socket e come si scopre quanto e' entrato: LO da' l'ultimo JSON trasmesso, non lo stato corrente, e durante IN e' escluso

Come chiuderli senza aspettare

Il proxy di analisi risponde da solo ai primi sette: basta puntarci il Dev Kit di PayPrint e osservare che cosa manda la loro applicazione, comprese le pause che tiene fra un comando e l'altro - non tutti pero': le causali dei movimenti e la stampa di immagini restano aperte anche dopo, mentre terminatore e pause li ha chiariti PayPrint e sul simulatore restano da provare. Degli ultimi sei, quattro hanno ora una risposta scritta del fornitore e amountPaid lo definisce il manuale; quello che resta richiede una macchina vera, perche' riguarda che cosa fa il dispositivo in condizioni che il simulatore non sa riprodurre - resto insufficiente, incasso interrotto a meta', messaggio spontaneo. PayPrint ha offerto il collegamento alla sua macchina.

9. Innesto in un gestionale esistente

Questo capitolo riassume l'analisi svolta su **Giano** (soluzione Neo.s.ln), il nostro gestionale per la ristorazione. Giano pilota già una cassa automatica di marca **VNE**: il pagAmico va *accanto* a quella, non al suo posto. L'analisi completa, con i riferimenti riga per riga, sta in docs/analisi-giano-vne-vs-pagamico.md; qui c'è quello che serve a chi lavora sulla libreria.

9.1 La superficie da coprire è piccola

Il livello VNE di Giano implementa **29 comandi**. Nel flusso di vendita ne girano **cinque**: apertura dell'incasso, interrogazione, annullo, elenco dei pagamenti pendenti, chiusura sessione. Un sesto - la versione macchina, usata come *ping* - vive fuori dalla vendita, nell'avvio hardware e nelle pagine di stato. Tre stanno in rami irraggiungibili della finestra di pagamento, uno solo è invocato dalla finestra di prova hardware, e i restanti diciannove - prelievi, svuotamenti, chiusura cassa, storici, configurazione - non hanno un solo chiamante: è copertura di protocollo scritta e mai usata.

Non manca quasi niente, quindi. Il lavoro non è aggiungere comandi: è **tradurre un modello in un altro**.

9.2 I due modelli a confronto

	VNE (oggi)	pagAmico
trasporto	REST/JSON, un POST per operazione	socket TCP persistente
stato della transazione	sulla macchina , con un identificativo interrogabile	nel client , in memoria
avanzamento	interrogazione ogni 500 ms	notifiche spinte durante l'incasso
concorrenza	libera: più thread, più richieste	un comando alla volta, serializzati
transazioni aperte	una coda, elencabile per identificativo	una sola, della connessione
dopo un riavvio	si enumerano i pendenti e si annullano alla cieca	resta solo LO, l'ultimo JSON trasmesso

Tre membri dell'interfaccia hardware di Giano - interrogazione, elenco pendenti, chiusura sessione - **non hanno alcun corrispondente**; contando anche le due modalità di rifornimento e lo stato cassa completo, i membri da **eliminare** sono sei. Il thread di interrogazione non si adatta: si riscrive.

9.3 Il punto che fa più male: l'annullo cambia segno contabile

Giano annulla i pagamenti chiedendo alla macchina di *trattenere* quanto già incassato, e registra l'operazione come **annullata**: nessuna vendita, il denaro resta nel dispositivo. Lo fa in tre chiamanti su quattro.

Il comando pagAmico che compie lo stesso gesto fisico è CM. Ma CM significa **incasso accettato e chiuso**, tanto che restituisce l'importo trattenuto (collectedAmount).

Non esiste una traduzione neutra

Stessa azione sul contante, esito contabile **opposto**. Portare il codice attuale di peso significa registrare incassi che non ci sono, oppure restituire denaro che andava trattenuto. È una decisione, e va presa prima di scrivere una riga di codice.

9.4 Compatibilità: non è un problema

Giano e' WPF su .NET Framework 4.7, con csproj in formato non-SDK e linguaggio fermo a C# 7.3. Sembrava il punto piu' spinoso dell'innesto; non lo e'.

Fatto	Conseguenza
la libreria dichiara gia' <code>net8.0;netstandard2.0;net47</code> e la DLL <code>net47</code> e' compilata	niente da ricompilare
le dieci dipendenze runtime sono gia' in <code>Neo/packages.config</code> alla versione identica (<code>System.Text.Json 10.0.2</code> in testa)	zero pacchetti nuovi, zero <i>binding redirect</i> nuovi
il vincolo C# 7.3 riguarda il codice di Neo , non un assembly referenziato	la libreria resta a <code>LangVersion latest</code>

Da qui la scelta: **DLL sotto** `ExternalReferences/`, non progetto sotto `ExternalProjects/`. Ricompilare la libreria in C# 7.3 costerebbe la rimozione dei *nullable reference types* su oltre un centinaio di punti della superficie pubblica - che sono l'unica documentazione formale di cosa puo' essere null in un'API che maneggia denaro - piu' i namespace a livello di file, le proprieta' `init`, i pattern relazionali e le *switch expression*. Riscrittura invasiva di codice collaudato, beneficio funzionale nullo.

Unica fragilita' segnalata: il framer usa `string.Contains(string, StringComparison)`, un overload **assente dal BCL di .NET Framework**. Compila oggi solo grazie alla combinazione `LangVersion=latest + System.Memory`; sostituirlo con `IndexOf(..., StringComparison.OrdinalIgnoreCase) >= 0` renderebbe il target `net47` indipendente dal compilatore.

9.5 Che cosa la libreria deve offrire a chi la innesta

Tre vincoli non evidenti dalla sola lettura dell'API, che chi integra scopre tardi se nessuno glieli dice. Sono documentati anche nel README, sezione *Come e' fatto il client*.

Vincolo	Perche' riguarda chi integra
a incasso aperto partono solo AN e CM	ogni altro invio - stato, display, stampa - lancia subito <code>PagAmicoCollectionOpenException</code> . Commit e annullo passano per una via laterale fuori dal mutex, dopo l'OK, uno solo per incasso. In Kotlin, se il chiamante dell'incasso viene cancellato, l'esito arriva a <code>onOrphanFrame</code>
il predicato dell'incasso lavora in due fasi	prima dell'OK un testo e' un rifiuto (<code>PagAmicoRejectedException</code> , o <code>PagAmicoBusyException</code>): non e' entrato denaro. Dopo, chiudono solo l'esito, AN e il CM finale; il resto va all'evento dei frame orfani , che puo' portare importi e va ascoltato. Corretto l'11 settembre (difetto D3)
gli incassi hanno sei cifre di centesimi	massimo 9.999,99 EUR per tutti e quattro - contanti, con timeout, POS, automatico - oltre il quale la libreria solleva un'eccezione prima di trasmettere . L'erogazione singola ne ha dieci: il tetto e' proprio dell'incasso. Serve un controllo esplicito a monte. Per i contanti PayPrint conferma il tetto; per il POS lo dice <i>maggiore</i> , in contrasto con il manuale, dove P0 ha sei cifre fisse (p. 17), e con la libreria (<code>PagAmicoCommands.cs:110</code>): punto aperto

9.6 Il collo di bottiglia e' nel gestionale, non nel box

Vale la pena dirlo perche' cambia le priorita' del lavoro. Oggi, quando la finestra di pagamento di Giano si chiude, l'unica informazione che attraversa il confine verso il resto dell'applicazione e' **un booleano**: annullato oppure no. Importo effettivamente incassato, resto erogato, resto *non* erogato, identificativo della transazione: niente di tutto questo esce, e niente viene mai scritto da nessuna parte.

Il `pagAmico` offre **piu'** dati del VNE - un importo non erogato in un campo dedicato invece che da ricavare per sottrazione, e un identificativo di movimento persistente sul dispositivo, riconciliabile a posteriori con lo

storico. Ma finche' quella riga resta un booleano, si perdono esattamente come si perdono oggi.

Conseguenza pratica

Sostituire il box senza allargare quel confine e' lavoro sprecato: i buchi contabili che esistono oggi resterebbero tutti, con un dispositivo diverso sotto. Il rovescio positivo e' che con il pagAmico l'esito e' un oggetto unico, quindi la decisione *finalizzo o no* si puo' prendere da un punto solo.

Prime Software S.r.l. - documento tecnico interno